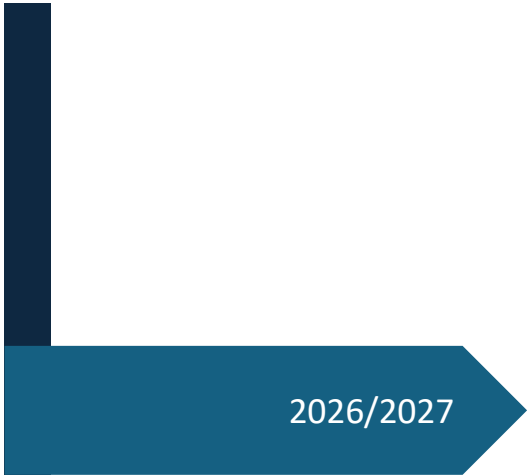
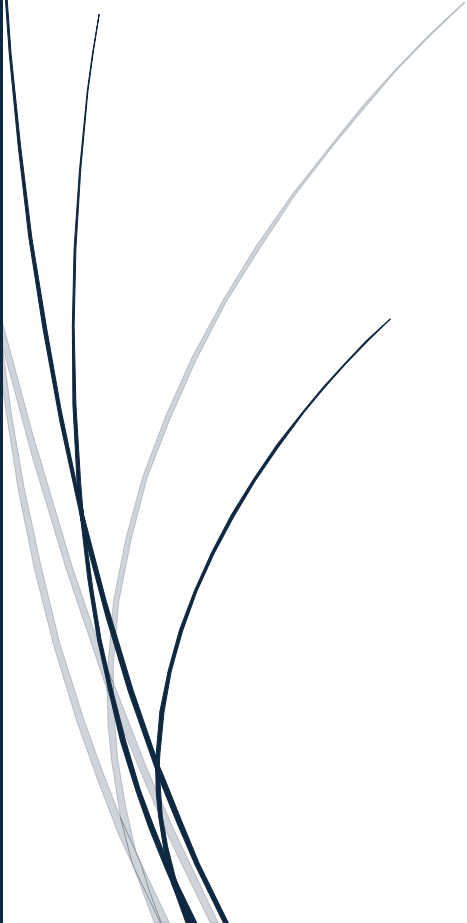


A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the text '2026/2027'.

2026/2027

Desarrollo Web en Entorno Servidor

DWES

Several thin, curved lines in dark blue and light grey originate from the bottom left and sweep upwards and to the right.

MOISÉS ALBERTO DOMÍNGUEZ CRUZ
IES LOS SAUCES – DAW2

TEMA 1 DESARROLLO WEB EN ENTORNO SERVIDOR

EJERCICIOS	2
1. Protocolos de comunicaciones: IP, TCP, HTTP, HTTPS.....	2
2. Modelo cliente-servidor	4
3. Métodos de petición HTTP/HTTPS.....	6
4. URI/URL/URN.....	7
5. Modelo multicapa.....	9
6. Front-end / Back-end	11
7. Tipos de web.....	12
8. Componentes de una aplicación web.....	14
9. Programas en cliente y servidor	15
10. Lenguajes lado servidor	17
11. Plataforma XAMPP.....	17
12. Uso de JVM y JDK.....	18
13. IDE más utilizados	18
14. Servidores HTTP/HTTPS	19
15. Apache HTTP vs Apache Tomcat.....	20
16. Navegadores	21
17. Generadores de documentación	22
18. Repositorios y control de versiones.....	22
19. Propuesta Entorno de Desarrollo	23
20. Propuesta Entorno de Explotación	23
21. CMS y ERP	23
22. Presentación de Arquitectura: Desarrollo elegido: MEAN	23
GLOSARIO DE TÉRMINOS RELACIONADOS CON DWES	24

EJERCICIOS

1. Protocolos de comunicaciones: IP, TCP, HTTP, HTTPS

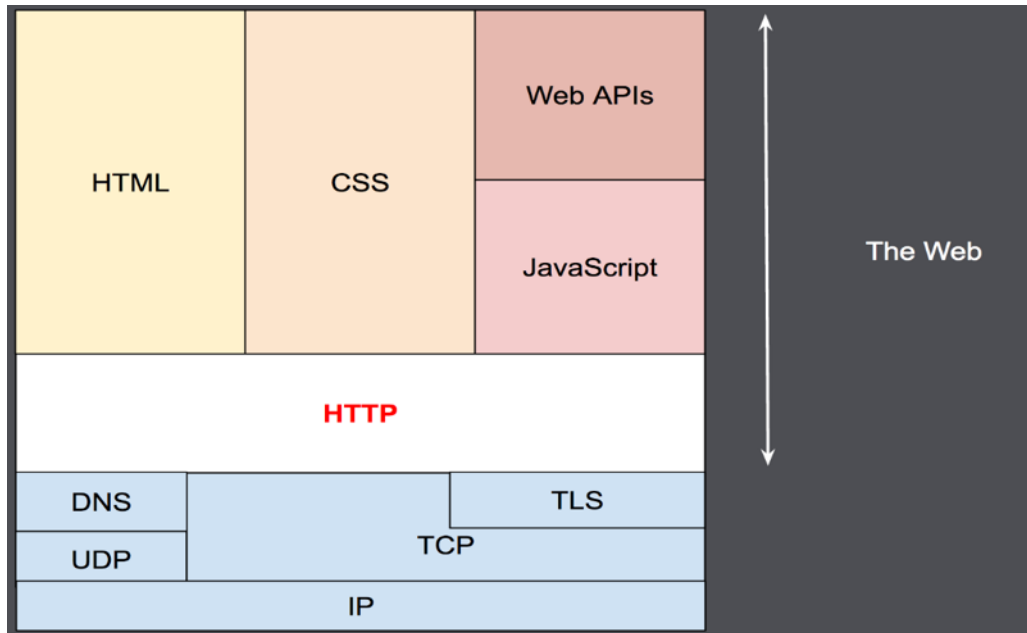
Los protocolos de comunicaciones son los encargados de determinar cómo deben circular los mensajes dentro de una red. Son conjuntos de normas que permiten la comunicación entre ordenadores, estableciendo la forma de identificación de estos en la red, la forma de transmisión de los datos y la forma en que la información debe procesarse.

IP → (Internet Protocol) es un conjunto de números que identifica, de manera lógica y jerárquica, a un dispositivo que utilice el protocolo o que corresponde al nivel de red del modelo TCP/IP.

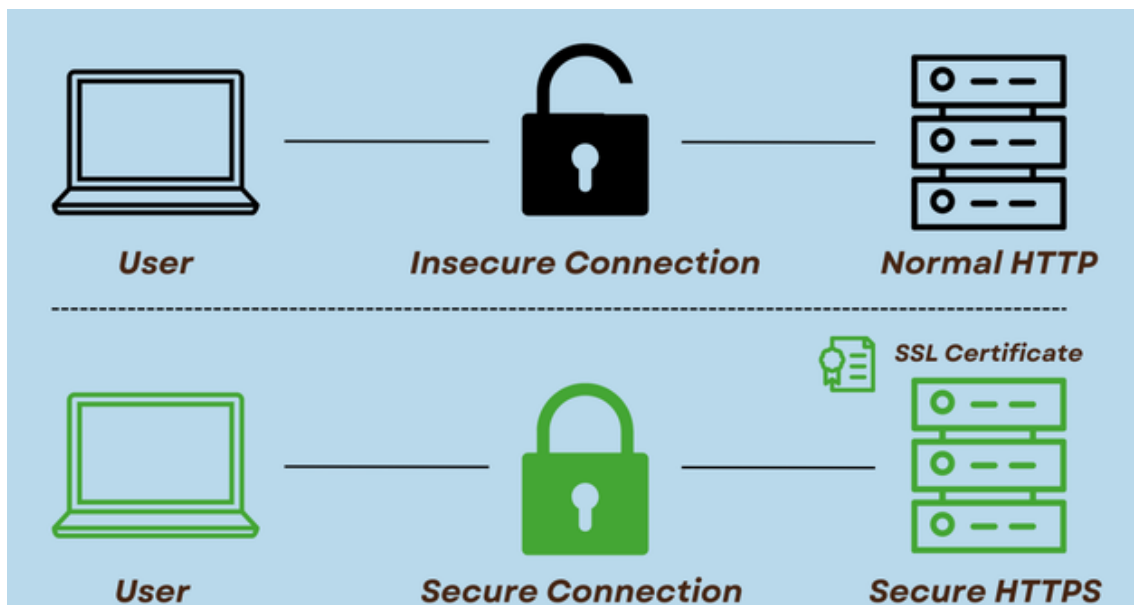


TCP → (Transmission Control Protocol) es un protocolo de red que permite que dos anfitriones (hosts) se conecten e intercambien flujos de datos. El rol de TCP es garantizar que los paquetes se entreguen de forma confiable y sin errores, con un sistema de recibo que permite al cliente y servidor garantizar la recepción mutua de datos.

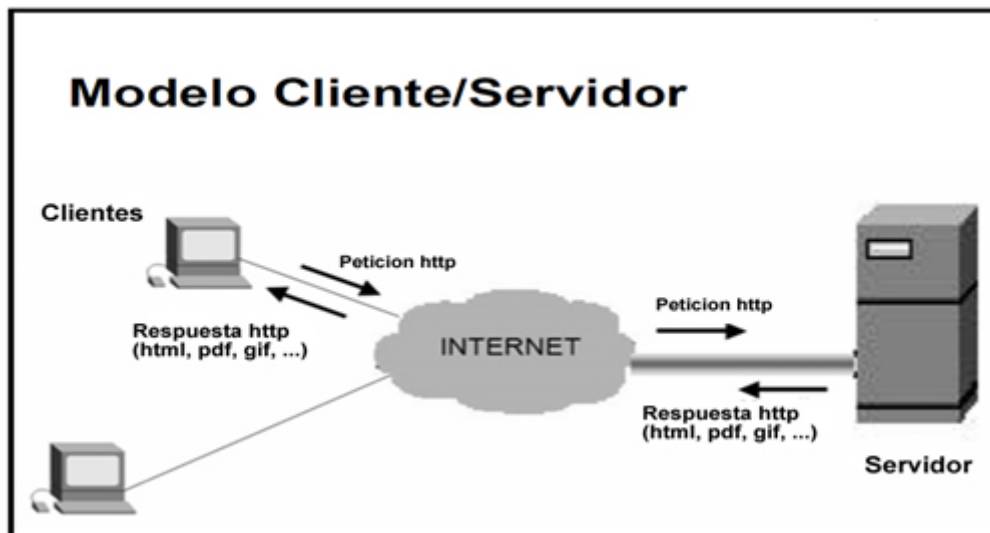
HTTP → (Transmission Control Protocol) es un protocolo de red que permite que dos anfitriones (hosts) se conecten e intercambien datos. El rol del protocolo TCP es garantizar que los paquetes se entreguen de forma confiable y sin errores, con un sistema que permite al cliente y servidor garantizar la recepción mutua de datos (puerto 80).



HTTPS → (Hypertext Transfer Protocol Secure) es igual que el protocolo HTTP, pero esta se transmite de forma segura. Esto se consigue mediante un cifrado basado en SSL/TLS para crear un canal cifrado entre cliente y servidor (puerto 443).



2. Modelo cliente-servidor



La arquitectura cliente-servidor es un modelo de diseño de software distribuido en dos partes, el cliente y el servidor, que se reparten las tareas. El cliente realiza peticiones al servidor y este le da las respuestas.

Se utiliza el protocolo HTTP/HTTPS tanto en la petición como en la respuesta.

Tipos de arquitecturas cliente servidor:

- **Arquitectura de dos capas**

Se utiliza para describir los sistemas cliente servidor en donde el cliente solicita recursos y el servidor responde directamente a la solicitud con sus propios recursos.

- **Arquitectura de tres capas**

Existe un nivel intermediario en la arquitectura generalmente compartida por un cliente que es el que solicita los recursos equipados con una interfaz de usuario o mediante un navegador web.

La capa del medio es denominada software intermedio cuya tarea es proporcionar los recursos solicitados pero que requiere de otro servidor para hacerlo. La última capa es el servidor de datos que proporciona al servidor de aplicaciones los datos necesarios para poder procesar y generar el servicio que solicitó el cliente en un principio.

- **Arquitectura N capas**

La arquitectura anterior, en tres niveles, es potencialmente una arquitectura en N capas ya que puede estar compuesto por N servidores donde cada uno de ellos muestra un servicio específico.

¿Cuál es su relación con las aplicaciones web?

Cualquier aplicación web que queramos desarrollar va a necesitar utilizar un servidor que haga uso del protocolo http o https, y un cliente que realice la petición, es decir, el modelo cliente-servidor.

El cliente (Navegador) hace una petición de una página al servidor web. Este servidor web tiene código programado en un lenguaje de programación (PHP). El servidor procesa este documento y el resultado es la devolución en formato HTML al cliente.

Enlace de interés: <https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor>

3. Métodos de petición HTTP/HTTPS

GET: Solicitar y obtener un recurso al servidor.

POST: Enviar datos al servidor para crear o procesar un recurso (ej. enviar un formulario).

Otros métodos de petición conocidos:

PUT / PATCH: Actualizar recursos totales o parciales.

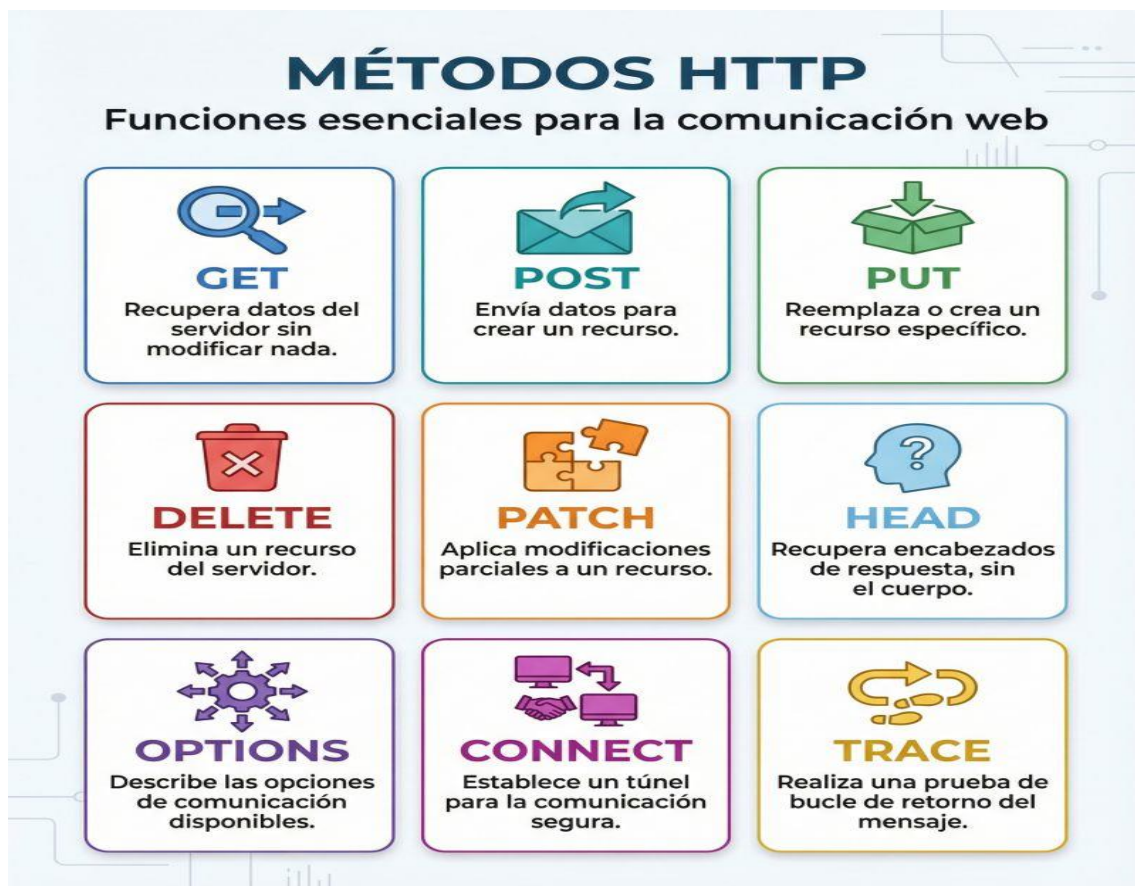
DELETE: Eliminar un recurso del servidor.

HEAD: Solicita una respuesta.

TRACE: Solicita al servidor un mensaje de vuelta.

OPTIONS: Devuelve los métodos HTTP para una URL específica.

CONNECT: Utilizado para acceder a un host.



4. URI/URL/URN

URI un sistema que ha sido diseñado para nombrar y localizar los recursos de la web de manera unívoca.

La estructura genérica de un URI se describe a continuación:

[protocolo][nombre_del_equipo][ruta_directorios][archivo]

Protocolo: También llamado esquema URL, este componente especifica el protocolo que se requiere utilizar para acceder al documento o recurso.

Nombre del equipo: (Habitualmente un nombre de dominio o una dirección IP) donde el contenido está alojado.

Ruta de directorios: Una secuencia de directorios separados por barras que define la ruta a seguir en la computadora para llegar al archivo.

Archivo: El nombre del archivo que contiene al recurso.

Ejemplo de un URI con todas sus partes etiquetadas:

Protocolo Nombre de la computadora Directorios Archivo
http:// www.htmlquick.com /reference/tags/ a.html

No es requerida la utilización de todas estas componentes para conformar un URI válido. Muchos URIs pueden tener sólo algunas de estas partes presentes (o ninguna) y aún ser válidas.

Algunos ejemplos de URIs válidos:

- Nombre de la computadora Directorios
195.156.123.1 /backup/ (sin protocolo; sin archivo)
- Protocolo Nombre de la computadora
http:// yeahyeah.com / (sin directorios; sin archivo)
- Directorios Archivo
/john/trabajo/ reporte.odt (sin protocolo; sin nombre de la computadora)
- Archivo
index.html (sin protocolo; sin nombre de computadora; sin directorios)
- / (sin protocolo; sin nombre de computadora; sin directorios; sin archivo)

Su relación con HTTP es directa, ya que las URL indican al navegador a qué servidor y qué archivo exacto solicitar mediante peticiones HTTP.

URI, URN, URL ¿qué son?



```
graph TD; URI[URI] --> URL[URL]; URI --> URN[URN];
```

URI ("Uniform Resource Identifier")
Secuencia de caracteres que permite identificar a un recurso.
Puede usarse como localizador del recurso, como nombre o como ambos.

URL ("Uniform Resource Locator")
Es un URI que permite localizar a un recurso. Incluye un protocolo.
`http://www.misitioweb.com`
`mailto:direccion@dominio.com`

URN ("Uniform Resource Name")
Es un URI que provee un nombre para un recurso.
`urn:uuid:6e8bc430-9c3a-11d9-9669-0800200c9a66`
`urn:ISSN:0167-6423`

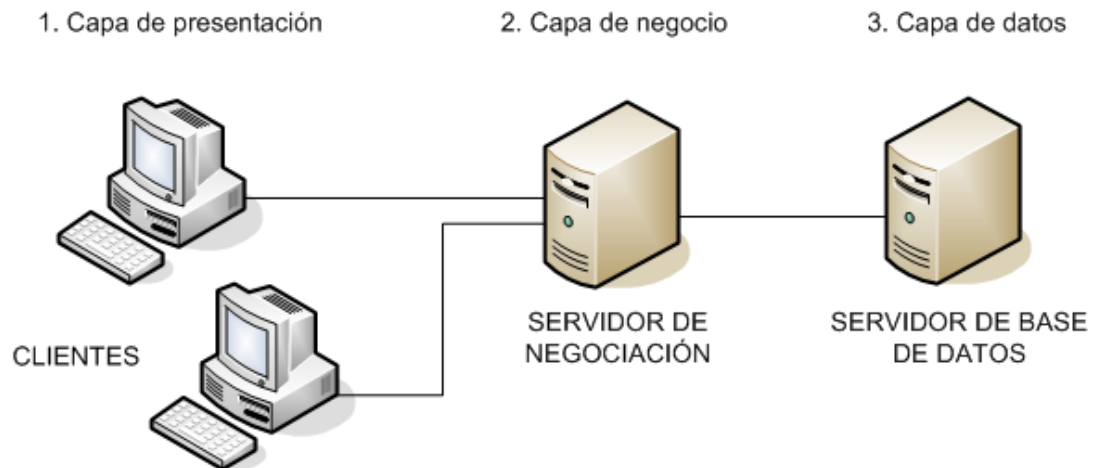
ejemplos



PROGRAMACIÓN DESDE CERO

5. Modelo multicapa

El modelo de desarrollo de aplicaciones multicapa divide el software en capas: Capa de Presentación (interfaz de usuario), Capa de Lógica de negocio (procesamiento de reglas y servidor web) y Capa de Datos (Bases de datos). Facilita el mantenimiento, seguridad y escalabilidad.



- Capa de presentación:

Presenta el sistema al usuario, le comunica la información y captura la información del usuario en un proceso. Esta capa se comunica únicamente con la capa de negocio. Sus características principales son:

- Recoge la información del usuario y la envía al servidor (cliente).
- Manda información a la capa de proceso para su procesado.
- Recibe los resultados de la capa de proceso.
- Generan la presentación.
- Visualizan la presentación al usuario (cliente).

- Capa de negocio:

Es donde residen los programas que se ejecutan, reciben las peticiones del usuario y se envían las respuestas tras el proceso. Esta capa se comunica con la capa de presentación, para recibir las solicitudes y presentar los resultados. Si la aplicación se comunica con otros sistemas que actúan en conjunto, lo hace mediante esta capa. También se comunica con la capa de datos para obtener información existente o ingresar nuevos datos. Sus características principales son:

- Recibe la entrada de datos de la capa de presentación.
- Interactúa con la capa de datos para realizar operaciones.
- Manda los resultados procesados a la capa de presentación.

- Capa de datos:

Aquí residen los datos y es la encargada de acceder a los mismos. Está formada por uno o más gestores de bases de datos que realizan todo el almacenamiento de datos, reciben solicitudes de almacenamiento o recuperación de información desde la capa de negocio. Para ello se utilizan APIs como pueden ser JDBC, de bajo nivel para acceder a las diferentes fuentes de datos.

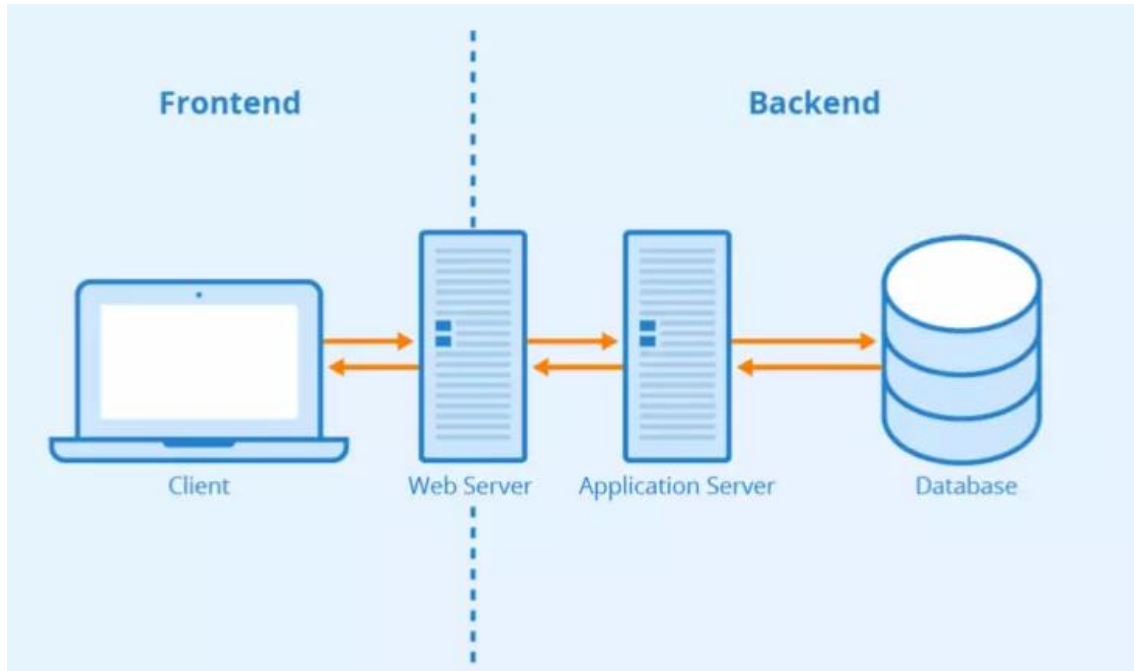
Sus características principales son:

- Almacena los datos.
- Recupera datos.
- Mantiene los datos.
- Asegura la integridad de los datos.



6. Front-end / Back-end

Front-end incluye lo que se ejecuta en el navegador del cliente (UI/UX, diseño, interactividad). Back-end gestiona el lado del servidor, es invisible para el usuario e incluye bases de datos, lógica de aplicación y autenticación.



Enlace de interés: https://es.wikipedia.org/wiki/Front_end_y_back_end

7. Tipos de web

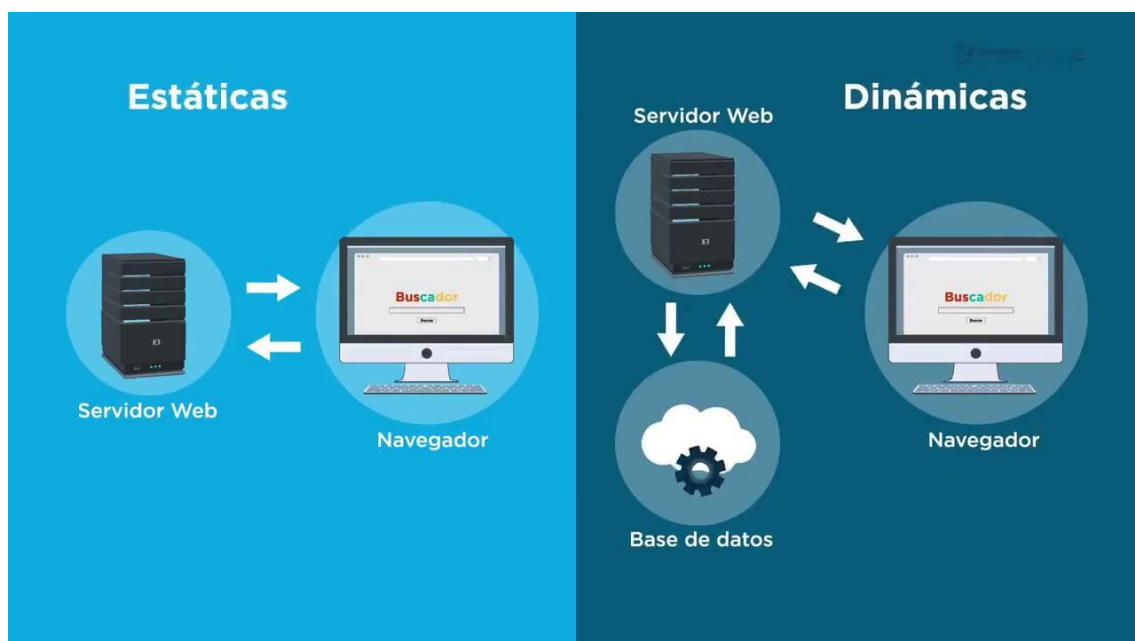
Estática: Es una página web en la cual el contenido no varía. El usuario se limita a obtener dicha información sin poder interactuar con la página web.

Una página web estática está compuesta por archivos HTML individuales por cada página que son pre generados y presentados al usuario a través del navegador de la misma forma.

Dinámica: Pagina web que permite crear aplicaciones dentro de la propia web, dando una mayor interactividad con el cliente.

Una página web dinámica incluye aspectos que se caracterizan por la interactividad y la funcionalidad, por ejemplo, los usuarios pueden interactuar con la información que se presenta en la página gracias a las instrucciones creadas a través de los lenguajes de programación y la base de datos sobre la que está construida.

Los sitios web dinámicos basan su comportamiento y funcionalidad en dos tipos de programación, front-end (del lado del cliente) y back-end (del lado del servidor). Las instrucciones del lado del cliente es código JavaScript que se ejecuta en el navegador. Mientras que las instrucciones que se ejecutan del lado del servidor son instrucciones escritas en lenguajes de scripting o programación, como .Net, PHP, Java... y que son ejecutadas para crear lo que el usuario ha solicitado en su interacción con la página.



Aplicación Web: Conjunto de herramientas que los usuarios pueden utilizar accediendo a un servidor web mediante un navegador, dicha página almacenara la información del usuario en su servidor.

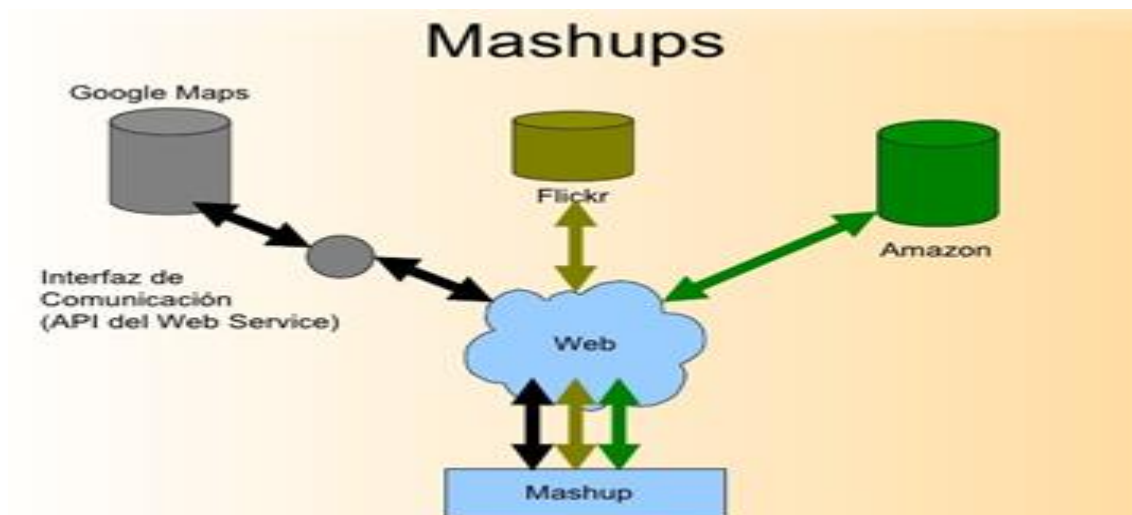
Las aplicaciones web son populares debido a lo práctico del navegador web como cliente ligero, a la independencia del sistema operativo, así como a la facilidad para actualizar y mantener aplicaciones web sin distribuir e instalar software a miles de usuarios potenciales.

Se pueden dividir en tres capas:

- Capa del navegador o presentación
- Capa del servidor o negocio
- Capa de persistencia o de datos.

Mashup: Es una aplicación web híbrida que combina servicios de diversas páginas web en una sola para ofrecer un nuevo servicio, se basan en utilizar contenido de terceros a través de fuentes o interfaces públicas.

Las empresas pueden crear experiencias innovadoras sin que los usuarios tengan que salir de sus páginas web. A su vez, las fuentes de mashup logran una mayor visibilidad que si sus servicios solo se ofrecieran a través de su página web.



8. Componentes de una aplicación web

1. El Cliente

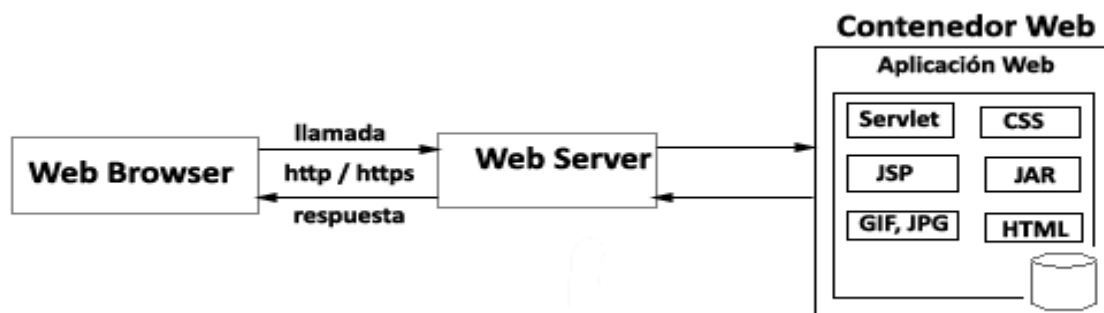
- Es la parte visible para el usuario.
- Se ejecuta directamente en el navegador web (como Chrome o Firefox).
- Utiliza tecnologías como HTML (estructura), CSS (diseño) y JavaScript (interactividad).

2. El Servidor

- Es el cerebro oculto que procesa la información.
- Se ejecuta en un ordenador remoto o servidor web desde el protocolo HTTP O HTTPS y en sus puertos 80 o 443.
- Recibe las peticiones del cliente, ejecuta la lógica de negocio y devuelve una respuesta.
- Utiliza lenguajes como Python, Node.js, PHP o Java.

3. Sistema gestor de base de datos

- Es el sistema encargado de almacenar, organizar y consultar los datos de la aplicación.
- Guarda información como usuarios, contraseñas o registros.
- Utiliza gestores como MySQL, PostgreSQL o MongoDB.



Enlace de interés: <https://platzi.com/blog/que-son-aplicaciones-web/>

9. Programas en cliente y servidor

Ciente: Se utilizan navegadores —Google Chrome, Mozilla Firefox, Safari, etc.—, que se comunican con los servidores e interpretan lenguajes tipados como HTML y CSS, y JavaScript o **TypeScript**. (en rojo los no vistos).

Lenguajes y tecnologías del lado de cliente
HTML: Lenguaje de marcas que indica la estructura de los contenidos de una página.
CSS: Da estilo a las páginas añadiendo márgenes, colores o tipografías.
JavaScript: Lenguaje que crea pequeños programas que realizan acciones dentro de una página web. Se insertan en HTML mediante la etiqueta <code><script></code> .

Servidor: Ejecuta procesos de negocio y acceso a datos. Lenguajes: PHP, Java, C#, Ruby, Node.js.

Lenguajes y tecnologías del lado del servidor
CGI: El sistema más antiguo para páginas dinámicas. Algo desfasado en la actualidad. Se suele utilizar Perl, aunque también en C, C++ o Visual Basic.
Perl: Lenguaje interpretado y extensible, se conecta muy bien con otros lenguajes.
ASP: Tecnología de Microsoft para páginas dinámicas de servidor. Se escribe en la propia página web con VBScript o JScript (Javascript de Microsoft).
PHP (Hypertext Preprocessor): Lenguaje rápido e independiente de la plataforma con una gran librería de funciones y una extensa documentación.
JSP (JavaServer Pages): Tecnología que permite crear páginas de servidor en lenguaje Java. Es multiplataforma.

Base de datos: Almacenan información necesaria para las aplicaciones. Los sistemas gestores de bases de datos las administran mediante queries. Ejemplos son MariaDB, MySQL o MongoDB, que se diferencia de las anteriores en que su almacenamiento de datos no es relacional.

Categoría	Descripción	Instrucciones
DDL—Data Definition Language	Especifican el esquema de la base de datos.	-create -alter -drop -rename
DML — Data Manipulation Language	Modifican la información almacenada.	-insert -update -delete -select
DCL —Data Control Language	Controlan los permisos de acceso a los datos.	-grant -revoke
TCL — Transaction Control Language	Llevan a cabo grupos de tareas.	-commit -rollback -savepoint

10. Lenguajes lado servidor

PHP: De los más utilizados actualmente, es de código abierto y muy rápido. Es fácil de aprender y multiplataforma. Permite establecer conexión con todo tipo de bases de datos, como Oracle o MySQL. El servidor, no obstante, no delega tareas al lado del cliente.

Java / C#: Altamente implantados en entornos corporativos, muy robustos y tipados.

Node.js (JavaScript): Alta adopción moderna para operaciones asíncronas y de tiempo real.

Python: Creciente popularidad gracias a la inteligencia artificial y a frameworks como Django.

11. Plataforma XAMPP

Paquete de software libre que agrupa de forma preconfigurada un servidor web Apache, base de datos MariaDB, y los lenguajes PHP y Perl (en sistemas multiplataforma "X").

Permite crear entornos de desarrollo web locales muy rápidamente.

componentes principales de XAMPP:

Apache: Es el servidor web más popular en internet, Apache es la aplicación de servidor web que procesa y entrega el contenido web al cliente.

MySQL: Es el sistema gestor de base de datos más popular y de código abierto. Está implementado en websites de aficionados hasta plataformas profesionales como WordPress. Cada aplicación web, ya sea simple o compleja, requiere una base de datos para almacenar los datos que recoge.

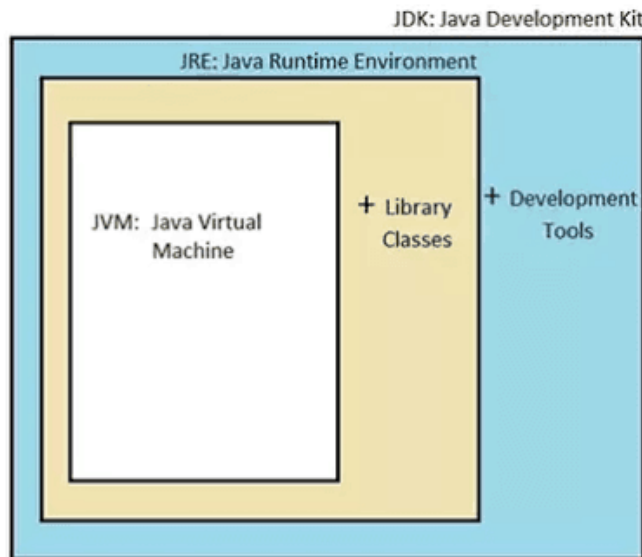
PHP: Es un lenguaje de script del lado del servidor que hace funcionar algunas de las páginas web más famosas del mundo, como WordPress o Facebook. Es de código abierto, relativamente fácil de aprender, y funciona perfectamente con MySQL, lo que lo ha hecho una opción muy popular para los desarrolladores web.

Perl: Es un lenguaje de programación dinámico y de alto nivel que se usa ampliamente en la programación de redes, la administración de sistemas, etc. Aunque menos popular para el desarrollo web, Perl tiene un montón de aplicaciones específicas.

12. Uso de JVM y JDK

El JDK (Java Development Kit) contiene los compiladores y librerías, siendo estrictamente necesario en el entorno de desarrollo para programar.

La JVM (Java Virtual Machine) es la máquina que ejecuta el código, siendo obligatoria tanto en el entorno de desarrollo como en el de explotación (servidor en producción).



$JDK = JRE + \text{Development Tools}$

$JRE = JVM + \text{Library Classes}$

13. IDE más utilizados

Visual Studio Code (el más popular y versátil, multilenguaje), IntelliJ IDEA (dominante en desarrollo Java), PhpStorm (líder de pago para PHP) y Eclipse/NetBeans (clásicos, perdiendo algo de mercado).

Todos ellos soportan múltiples lenguajes de programación como Java, PHP, C++, Python..., aunque algunos deben ser descargados vía plugin. También suelen incluir un intérprete o un compilador de código.

14. Servidores HTTP/HTTPS

Nginx (alto rendimiento, servidor de proxy inverso, gran implantación actual), Apache HTTP (muy extendido, uso de módulos .htaccess) e Cloudflare (red de entrega de contenido, servicios de seguridad de Internet y servicios de servidores de nombres de dominio distribuidos).

Web Servers

Most popular web servers

© W3Techs.com	usage	change since 1 August 2026
1. Cloudflare Server	30.9%	+1.9%
2. Nginx	30.9%	-0.6%
3. Apache	22.0%	-1.1%
4. LiteSpeed	14.5%	-0.5%
5. Node.js	7.2%	+0.4%

percentages of sites

Fastest growing web servers since 1 August 2026

© W3Techs.com	sites
1. Cloudflare Server	358.3
2. Node.js	80.5
3. Google Servers	7.7

daily increase of number of sites
per million

15. Apache HTTP vs Apache Tomcat

Apache HTTP es un servidor web generalizado de contenidos (estáticos y dinámicos como PHP).

Apache Tomcat es un contenedor de servlets específicamente diseñado para desplegar aplicaciones web desarrolladas en Java (JSP/Servlets).

Algunas diferencias son:

- La diferencia fundamental es que Tomcat proporciona contenido dinámico en Java, mientras que el propósito principal del servidor web Apache es simplemente servir contenido estático como HTML, imágenes, audio y texto.
- Tomcat se creó específicamente para aplicaciones Java, mientras que Apache es un servidor HTTP de propósito general. Puedes utilizar Apache junto con diferentes lenguajes de programación (PHP, Python, Perl, etc.) con la ayuda del módulo de Apache apropiado al lenguaje.
- En cuanto a velocidad, Apache HTTP es más rápido que Apache Tomcat en lo que a las páginas web estáticas se refiere.

Enlaces de Interés:

[Apache Tomcat vs Apache HTTP](#)

[Apache HTTP](#)

[Apache Tomcat](#)

16. Navegadores

Google Chrome (líder absoluto), Safari (dominante en ecosistema Apple), Microsoft Edge (ganando usuarios corporativos) y Mozilla Firefox (basado en software libre y privacidad).



Google Chrome: Rápido, sencillo e intuitivo. Tiene una fuerte seguridad, con actualizaciones automáticas. Como inconveniente consume demasiados recursos del PC.

Safari: Es el navegador que se suele utilizar en los Mac. Su sencillez le da estabilidad y eficiencia, aunque tampoco es el más rápido. La falta de actualizaciones hace que tenga menos seguridad.

Mozilla Firefox: Es uno de los más veteranos. Tiene actualizaciones constantes y su funcionamiento es estable. Es fácil de personalizar con extensiones.

Microsoft Edge: Es la alternativa más modernizada de Microsoft a Internet Explorer. Está más actualizado y es más rápido. Consume pocos recursos del PC.

Opera: También es veterano. Similar a Google Chrome, con un apartado para guardar páginas web. Tan personalizable como Chrome o Firefox.

Internet Explorer: Está actualmente obsoleto y por lo tanto tiene muchos problemas de seguridad. Su velocidad tampoco destaca.

17. Generadores de documentación

(Esto no lo has explicado)

18. Repositorios y control de versiones

Los programas para control de versiones son un conjunto de aplicaciones que gestionan ágilmente los cambios en el código fuente de los programas y poder revertirlos.

- **Git:** De las mejores herramientas de control de versiones disponible en el mercado actual. Es un modelo de repositorio distribuido compatible con sistemas y protocolos existentes como HTTP, FTP, SSH y es capaz de manejar eficientemente proyectos pequeños a grandes.
- **CVS:** Es otro sistema de control de versiones muy popular. Es un modelo de repositorio cliente-servidor donde varios desarrolladores pueden trabajar en el mismo proyecto en paralelo. El cliente CVS mantendrá actualizada la copia de trabajo del archivo y requiere intervención manual sólo cuando ocurre un conflicto de edición.
- **Mercurial:**
- **Monotone:**

19. Propuesta Entorno de Desarrollo

20. Propuesta Entorno de Explotación

21. CMS y ERP

22. Presentación de Arquitectura: Desarrollo elegido: MEAN

GLOSARIO DE TÉRMINOS RELACIONADOS CON DWES

Protocolos TCP/IP. Socket: TCP/IP es la suite de reglas de comunicación en red. Un Socket es un punto de conexión en red que empareja una dirección IP y un puerto.

Protocolo HTTP/HTTPS: Reglas de transferencia para hipertexto en la web. HTTPS incluye cifrado TLS.

HTML: Lenguaje de marcado para la estructura de la web.

XML: Lenguaje de marcado extensible para definir reglas de datos.

JSON: Formato de texto ligero y fácil de leer usado para intercambio de datos modernos (APIs).

Lenguajes embebidos en HTML: Lenguajes como PHP que se pueden incrustar entre etiquetas de HTML para generar la web final antes de enviarla.

Arquitecturas y Frameworks: Las arquitecturas (ej. MVC) definen el patrón de diseño. Los frameworks (ej. Laravel, Spring) son entornos de trabajo prefabricados que agilizan programar bajo esa arquitectura.

ERP / CMS: Software web empresarial general (ERP) o software web gestor de publicaciones (CMS).

PHP: Acrónimo recursivo de "PHP: Hypertext Preprocessor". Lenguaje para crear webs dinámicas desde el servidor.

IDE / Navegador / Repositorio: Herramienta integral de desarrollo (IDE), visor web del cliente (Navegador) y base de datos de control del código fuente de un proyecto (Repositorio).

Entornos de Desarrollo / Explotación: El desarrollo es el entorno local donde el programador prueba errores. La explotación (producción) es el servidor real público al que se conectan los clientes.

Gestión de configuración / Control de cambios: Metodologías sistemáticas para manejar las actualizaciones y corregir bugs en la aplicación sin romper lo existente.

Web Services y AJAX: Los Web Services proveen integración de máquinas vía web (REST/SOAP). AJAX permite interactuar con el servidor y refrescar información en la pantalla del usuario sin recargar toda la pestaña.

Aplicaciones basadas en microservicios / Multicapa: En lugar de tener una sola aplicación gigante (monolito multicapa), los microservicios dividen cada funcionalidad en pequeños programas que se conectan entre sí.

SaaS (Software as a Service): Distribución de software donde el cliente no lo instala, sino que lo consume bajo demanda alojado en la nube del proveedor.

Control de acceso y Validación de entrada: Procesos de ciberseguridad. Autenticar roles y filtrar que los datos que escribe el usuario no contengan código malicioso para el servidor (como inyecciones SQL).

Posicionamiento: Prácticas para que la aplicación web aparezca en los primeros puestos de los buscadores.

Historia y Filosofías de desarrollo: Evolución histórica de la web y uso de marcos de trabajo para entregar software funcional rápidamente.